Cross-Engine Learned Cost Models for SQL Workloads

Zero-Shot Engine Selection Using Machine Learning

Advanced Database Topics

(Winter 2026)

Members

Rinilkumar Parmar	parmar7f@uwindsor.ca
Krishal Modi	modi3b@uwindsor.ca
Aditiben Fatesinh Parmar	parmar3f@uwindsor.ca
Lisa Vijaybhai Mangnani	mangnan@uwindsor.ca
Janitha Reddy Salla	salla@uwindsor.ca
Krish Dimplekumar Thakkar	thakka4b@uwindsor.ca
Hiten Jagdishbhai Patil	patil46@uwindsor.ca

Dept. of Computer Science

University of Windsor

(Masters of Applied Computing)

Introduction

- Modern database systems increasingly adopt composable architectures with multiple specialized engines.
- The same SQL query can run on heterogeneous engines (e.g., PostgreSQL, DuckDB, SQLite) with very different performance.
- Current DB optimizers are engine-local and cannot choose the best engine for a query.
- This limitation is critical in healthcare and finance, where systems must support OLTP + analytics with strong scalability and reliability.
- Over 60% of enterprise applications use multiple database systems, increasing performance unpredictability and operational complexity.

University of Windsor

Problem Statement

- Existing query optimizers assume a fixed execution engine for SQL queries.
- The same SQL workload can have very different performance across engines like PostgreSQL, DuckDB, and SQLite.
- Query performance varies based on data size, joins, and execution strategies.
- Current systems lack an automatic, cost-based mechanism to select the best engine before execution.

University of Windsor

Objectives & Novel Contributions

Novelty Claim

- We break the traditional assumption that query optimization must be engine-local by introducing an engine-agnostic, learned decision layer that selects the optimal database engine before query execution.

Objectives

- Design an external, engine-agnostic abstraction for SQL execution across multiple DB engines.
- Extract pre-execution logical query features independent of engine internals.
- Learn engine-specific cost models to predict query execution time.
- Enable zero-shot engine selection without trial execution or manual heuristics.
- Evaluate improvements in latency and engine-selection accuracy against heuristic baselines.

University of Windsor

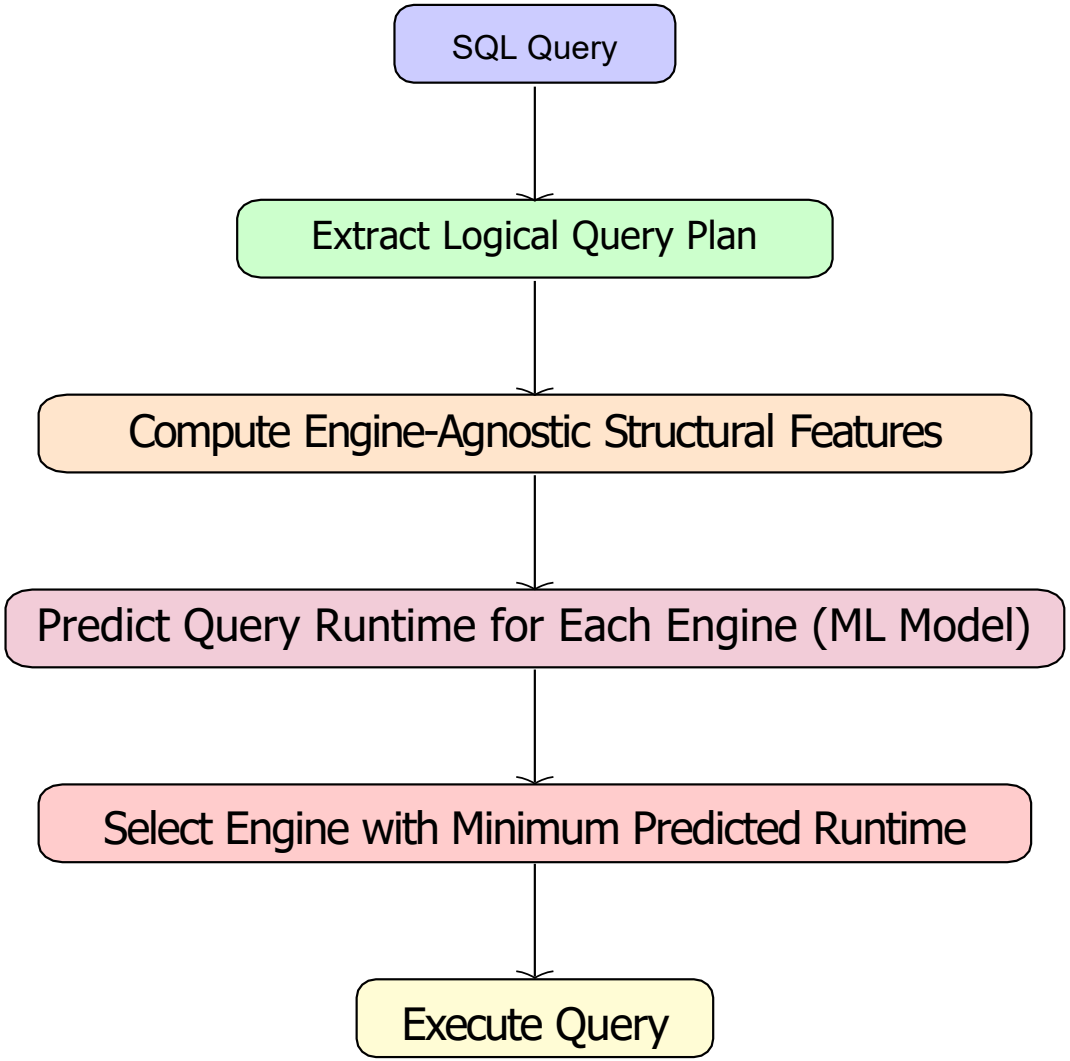
Literature Review (Related Work)

- Prior work has significantly advanced query optimization and learned cost modeling, but remains largely engine-local or manually configured in multi-engine settings

Work	Focus	Limitation
SIGMOD'20	Learned cost models	Limited to single-engine queries
VLDB'21	Polystore systems	Engine selection requires manual routing
CIDR'22	Adaptive query plans	Cannot adapt across multiple engines
SIGMOD'23	ML for DB tuning	Models tied to one engine; no cross-engine generalization

University of Windsor

System Architecture/Flow Diagram



University of Windsor

Methodology

- **Query Parser:** Receives SQL input and generates a logical query plan.
- **Feature Extractor:** Captures engine-independent features such as number of joins, filters, aggregations, and data size.
- **Prediction Model:** Machine-learning based cost model predicts execution time for each candidate engine.
- **Execution Layer:** Runs the query on the engine with the lowest predicted cost.
- **Feedback Module:** Collects runtime metrics to refine models over time.

University of Windsor

Methodology

➢ **Cost-Based, Engine-Agnostic Optimization**

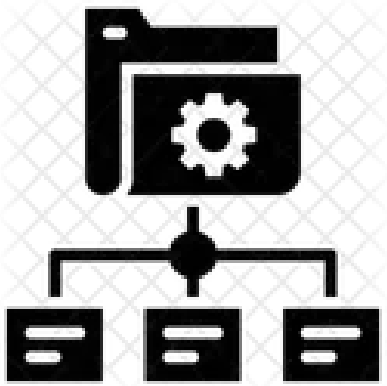
- Predicts query runtime across multiple engines and selects the fastest execution path.
- Extends traditional cost-based optimization from plan-level to cross-engine decision-making.

➢ **Logical–Physical Data Independence**

- Uses logical query plans instead of engine-specific physical plans.
- Enables engine-agnostic decisions, keeping SQL queries portable across multiple DBMSs.

➢ **Adaptive Execution**

- Adapts dynamically to changing workloads and data distributions.
- Uses feedback from executed queries to improve future engine-selection accuracy.



University of Windsor

Risk Analysis

➢ **Data Inconsistency**

- Validate query results across engines; maintain synchronized test datasets; apply checksums for critical queries.

➢ **Security Attacks**

- Use secure DB connections (SSL/TLS); restrict access to testing environment; follow best practices for credentials and data handling.

➢ **Tool / Model Limitations**

- Start with lightweight ML models (linear regression, random forest); gradually extend to more complex models; cross-validate to prevent overfitting.

University of Windsor

Implementation Plan & Timeline

Implementation Stack

Databases: PostgreSQL, DuckDB, SQLite

Language / Tools: Python, scikit-learn, Pandas

Dataset: TPC-H benchmark + synthetic query variants

Machine Learning Models: Linear Regression, Random Forest (engine-specific cost prediction)

Week	Tasks
Week 1-2	Environment setup, DB configuration, dataset preparation
Week 3	Extract logical query plans, compute engine-agnostic features
Week 4-5	Train and validate ML cost models for each engine
Week 6	Implement engine-selection module and integrate system
Week 7	Run experiments, evaluate engine-selection accuracy and latency
Week 8	Fine-tune models, analyze results, prepare final report & presentation

University of Windsor

Real World Applications

- **Cloud Data Platforms:** Automatically pick the fastest engine across multiple services.

Example: User data on SQLite, analytics on DuckDB, fastest query chosen automatically.

- **Data Analytics:** Optimize queries based on workload type.

Example: Daily sales on SQLite, yearly trends on DuckDB.

- **Dashboards & Reports:** Ensure fast-loading charts and reports.

Example: Monthly sales dashboard updates instantly; complex yearly reports run on faster engine.

- **AI/ML Data Prep:** Speed up preprocessing for ML models.

Example: Small lookups on SQLite, heavy joins on DuckDB.

University of Windsor

References

- A. Strausz, N. Pardon, and I. Giurgiu, "A Learned Cost Model-based Cross-engine Optimizer for SQL Workloads," arXiv preprint arXiv:2506.02802, 2025.
- R. Heinrich, X. Li, and Z. Kaoudi, "Learned Cost Models for Query Optimization: From Batch to Streaming Systems," PVLDB, vol. 18, no. 12, 2025.
- R. Marcus, P. Negi, H. Mao, N. Tatbul, and T. Kraska, "Plan-Structured Deep Neural Network Models for Query Performance Prediction," PVLDB, vol. 12, no. 11, 2019.
- U. Pathak and A. Mankodi, "Redefining Cost Estimation in Database Systems: The Role of Execution Plan Features and Machine Learning," arXiv preprint arXiv:2510.05612, 2025.

University of Windsor

References

Thank You

University of Windsor